APMAN

François Cayre, Auguste Olivry

Phelma / 2A SEOC

2026–2027

Course objectives

1. To tackle real-world issues in programming, testing and benchmarking
2. To learn (quite) a few techniques in C
   Modules, genericity, iterators, meta-programming, generalization/specialization
3. To understand how a streamlined methodology brings *guarantees* to your code
4. To master complexity issues—in (applied) Computer Science *and* with experiments
5. To understand (and close!) the gap between the textbook and robust, fast code

You are given an environment that implements much of what these slides discuss:

- It's only good news on a higher level: that's a lot of real, *hardcore* C to digest.
- These slides describe the techniques we used as independently as possible.
  Sections "Setting the stage" and "Basic tools for data structures."
- Ask yourself/us how to make the most out of it! With the least effort!

Outline

Hashing (for data indexing)

A subjective history of *"Where did I put the damn keys?"*

A world of complexity

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking

`TODO`

Searching, by hand

Earliest alternative to linear search: hand-built index

- Mesopotamia
  Clay tablets organized in collections, lexical catalogues constructed *systematically*.
- Ancien Egypt
  Administrative records by category, scribes produce *ordered* word (and sign) lists.
  - Most often: single-level indexing
- Greco-Roman world: most notable example is the Library of Alexandria
  - Callimachus' *Pinakes*: huge catalogue by author *and* subject, with biblio info
  - Likely the first known database
  - "Practical" multi-level indexing
- Romans credited to leverage alphabetical order for organization and search
  - "Abstract" multi-level indexing
- Pre-Google Internet had collaborative construction of curated website collections
  - Still used on the Darknet today (darksites not discoverable by construction)

Searching, with a machine

Main problems with an index: built by hand, logarithmic up to index depth.
You may still have a long list of data to check once you reached an index leaf.

- Mid-1940's: *Associative access* (making the goal explicit)
  K. Zuse (Germany), V. Bush (USA).
  "What if those strange machines allowed to search data by its very contents*?"*
  - Goal: automatic (non-human) scribes.
- 1953: *Chaining hashtables* (closed addressing)
  H.P. Luhn.
  "Let's compute from the data itself *the number of the list where it belongs!"*
  - On an IBM 701: 2000 words of magnetic-drum memory, 72-bit machine words;
  - Full pointers too costly: have smaller indices into storage array instead;
  - That's basically our pointer arithmetics (to save on the pointers size.)

Searching, with a machine of today

- 1954: *Linear hashtables* (open addressing)
  G. Amdahl, E. McGraw, A. Samuel (IBM 701) + A. Ershov independently.
  "By the way, you could even have implicit *lists!"*
- Late 1960's: The word "hashing" eventually becomes standard terminology
  - Many variants explored: quadratic hashing, double-hashing, Cuckoo hashing, *etc.*
- 1986: *Robin-Hood hashing* (minimize inequalities to maximize robustness)
  P. Celis.
  "Let's have a linear table self-organize to maintain highest performance*!"*
  - Robustness? *RH maintains highest performance under a larger panel of workloads.*
- 2022: *SwissTable* (leverage insn-level, parallel probing)
  Abseil (company).
  "Let's have ultra-fast SIMD lookups with metadata *control bytes!"*
  - Robin-Hood on steroids (most lookups with a single probing insn)
  - Influential design (Rust's HashBrown table, *etc.*)

Outline

Hashing (for data indexing)
A subjective history of *"Where did I put the damn keys?"*
A world of complexity

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking

TODO

The big picture (searching)

Applications fall in two categories:

1. If you are only concerned with *exact search*, go with hashtables
   "What is the value of this key?"
   Hashtables are unbeatable at this game (amortized constant-time)
2. If you need *interval search*, go with ordered search trees
   "What are the keys in this interval?" (spatial data, ecological data, *etc.*)
   - *Lower-bound* search primitive.
   - The lower bound in the data may or may not be that of the search interval
   - Easy to search for absence or presence+value of one specific key
     To instantiate exact search for comparison purposes.

That's a lot of ground to cover!

The big picture (hashing)

We are *not* doing crypto!

- Hash data to produce secure digests (Keccak, *etc.*)
- Much more stringent mathematical requirements
  Collision resistance, pre-image resistance, second pre-image resistance.

But we may still have adversaries:

- DoS (Denial-of-Service) attack on routers
- Targeted weakness was the router internal hashtable
- Hackers triggering catastrophic, degenerate linear-time search
  Poorly designed hash function allowed crafting the right attack sequence
- SipHash hash function specifically designed as a counter-measure

The big picture (hashing + searching)

- A hash is a source of pseudo-randomness derived from the key alone.

Search trees may use a hash in essentially two forms:

1. Cheap and efficient randomized balancing of a BST (treap)
   More complicated and stringent balancing with AVL trees, Red-Black trees.
2. Plain routing info inside the tree (hashtrie)
   Balancing unordered keys.

It is common practice for a hash-based search structure to *finalize* the hash.
The user must provide the hash function, which may be poorly designed:

- Add some internal layer of "hashing the hash"
- Must be fast (a few CPU insns): XOR-based bit-mixer, Fibonacci hashing

Cheap patch against entry-level contingencies, close to useless under attack.

- Gain operational reach and stability.

Outline

Hashing (for data indexing)

Setting the stage
- Preprocessor
- Debugging
- Error handling
- Memory management

Basic tools for data structures

Maps

Testing and benchmarking

TODO

What is it?

- Commands that (conditionnally) transform text, *prior to compilation*
  - `#include`, `#if`, `#else`, `#elif`, `#endif`, `#define`, `#ifdef`, `#ifndef`.
- Two output facilities, one to trigger failure:
  - `#warning "You␣have␣been␣warned"`: compilation continues after printing
  - `#error "teH␣iMPosSiBLe␣HaPpEneD!"`: compilation fails after printing
- Two operators (your life just changed):
  - `# symbol`
    will create *constant string* `"symbol"` (symbol "stringification")
  - `foo ## bar`
    will create *symbol* `foobar` (symbol concatenation)
  - One minor glitch: better used encapsulated, more later.
- What is actually compiled is the output of the preprocessor.
  - What about conditional copy-pastes that end up writing code for you?

Example

Crudeness of the preprocessor commands makes debugging them a little tedious:

```
1   #include <stdint.h> /* Integers and their bounds */
2
3   #if UINTPTR_MAX == UINT64_MAX
4   #warning "64-bit␣pointers.␣Now␣that's␣a␣machine!"
5
6   #elif UINTPTR_MAX == UINT32_MAX
7   #warning "32-bit␣pointers.␣I␣like␣little␣gadgets␣too!"
8
9   #else
10  #error "I␣will␣refuse␣to␣compile␣on␣alien␣hardware"
11  #endif
```

There's no `printf()`-like facility nor any C construct!
Except for (possibly parenthesized) logic/arithmetic testing with `#if` and `#elif`.

Symbols

- Preprocessor symbols may or may not be defined.
- When they are defined, they may or may not have a value.
  - And when they have a value, it may or may not be a *macro*.

This will eventually let us achieve C code abstraction w.r.t.

- Machine-dependent features (pointer size, SIMD extensions, *etc.*)
  - Isolate code for bleeding-edge hardware, preserve safe fallback for production.
- Lower-level implementation choices:
  - `set.h` → `map.h` → { `hashtable.h`, `bst.h`, `list.h` }
  - `set.h` will essentially build on `put()`/`get()` from `map.h` to implement union, membership testing, symmetric difference, *etc.*
    Any other structure providing `put()`/`get()` could be used.
  - `map.h` will factor out all possible code so the core algorithmic differences are isolated in `list.h`, `bst.h` and `hashtable.h`.

Preprocessor caveats (side-effects)

Suppose we have:

```
1  #define gen_abs(x) ((x)<0?-(x):(x))
```

If you use it with side-effects, a kitten will die:

```
1  int a = -3;
2
3  int gabs = gen_abs(a++);
4
5  /* Expands to:   int gabs = ((a++)<0?-(a++):(a++));
6
7     => The value of a is now -1, yet -2 was likely expected.
8  */
```

Preprocessor caveats (parentheses)

Suppose we have:

```
1  #define get_idx(p) p->idx
```

If you use it with pointer arithmetics, a kitten will die:

```
1  struct foo_t { int idx; };
2
3  struct foo_t *arr = calloc(42, sizeof *arr);
4  /* Fill arr as expected */
5  int id = get_idx(arr+2);
6
7  /* Expands to:   int id = arr+2->idx;
8
9     => Syntax error.
10 */
```

Preprocessor caveats (do and don't)

1. *Never* use a macro with side-effects in the argument(s);
2. Put parentheses around every parameter occurrence:

```
1    #define get_idx(p) (p)->idx
```

And around the macro body itself if it's to be used inside other expressions:

```
1    #define list_is_empty(list) ((list) == LIST_END_MARKER)
```

Incipit tragœdia.

Common pattern: Arch.-dependent constants

Suppose the start of some `src/list.c`:

```
1   #include <stdint.h> /* Integers and their bounds */
2   /* We may have other plans for NULL */
3   #if   UINTPTR_MAX == UINT64_MAX
4   /* Define a symbol with a constant value */
5   #define LIST_END_MARKER ((void*)0xffffffffffffffffULL)
6   #elif UINTPTR_MAX == UINT32_MAX
7   #define LIST_END_MARKER ((void*)0xffffffffUL)
8   #else
9   #error "Unsupported␣architecture"
10  #endif
11
12  /* Define a *macro* symbol, works on 32/64-bit machines */
13  #define LIST_END(list) ((list) == LIST_END_MARKER)
```

Common pattern: Arch.-dependent constants

Our `src/list.c` may continue like:

```
1   #include <stddef.h> /* size_t */
2
3   size_t list_length(list_t l) {
4     size_t len = 0;
5
6     for ( ; !LIST_END(l) ; l = l->next)
7       len++;
8
9     return len;
10  }
11
12  /* Plenty of code to finish implementing lists */
13
14  #undef LIST_END          /* Clean locally-defined symbols */
15  #undef LIST_END_MARKER
```

Common pattern: Arch.-dependent constants (cont'd)

What the compiler will automatically get for compilation on a 32-bit machine:

```
1    size_t list_length(list_t l) {
2      size_t len = 0;
3
4      for ( ; !((l)==(void*)0xffffffffUL) ; l = l->next)
5        len++;
6
7      return len;
8    }
```

Use `-E` compiler flag to get the preprocessor output.
Warning! This is very verbose (all the `#include`'d contents is there).
You have to anticipate for the correct expansion of macros and symbol values!

Common pattern: Conditional inclusion

Suppose a higher-level `include/structure.h`:

```
1   #if defined(TREE_STRUCT)
2
3   #include <structure/tree.h>
4
5   #elif defined(LIST_STRUCT)
6
7   #include <structure/list.h>
8
9   #else
10  #error "Unsupported/unspecified␣data␣structure"
11  #endif /* TREE_STRUCT, LIST_STRUCT */
12
13  /* Common code for all data structures */
```

Common pattern: Conditional inclusion

Now in some `code.c`, we'd certainly expect to be able to do:

```
1  #define LIST_STRUCT
2  #include <structure.h>
3  /* Will eventually include <structure/list.h> */
```

You may want to see `LIST_STRUCT` as a *parameter* of the *template* `structure.h`.
The idea is:

- to let the user specifiy an option (here: a data structure backend);
- to factor out the maximum amount of code among options;
- to have the preprocessor write the actual code for us.

This is the basic paradigm of metaprogramming: programming in program space.

- Scheme, Rust (and many others!) feature much more powerful such capabilities.
- The C preprocessor has internal limits (and arguably a primitive syntax!)

Common pattern: Templating (motivation)

Suppose you want a map to get the histogram of words (mutations) in a text (DNA).

- The map keys are the (unique) words, their value is the word count in the text.

We want a *generic* map template that could be *instantiated* more or less like:

```
1    #define key_t         char*
2    #define key_hash      word_hash
3    #define key_compare   wordptr_compare
4    #define value_t       size_t
5    #define tag           histogram
6    #include <structure/map.h>
```

- Expected result: After line 6, we have the `histogram_t` type and we can use `histogram()` (the *constructor*), `histogram_get()`, `histogram_put()`, *etc.*
- Encapsulating concatenation is mandatory due to the preprocessor internals:

```
1    #define CONCAT(foo,bar) foo ## bar
2    #define CAT(foo,bar)    CONCAT(foo,bar)
```

Common pattern: Templating

Now in `map.h`, we first define (*prologue*) and then clean (*epilogue*) local symbols:

```
1   /* Generic type name: T (map of any key/value types) */
2   #define T          CAT(tag,_t)    /* Symbolic prologue */
3   #define map_put    CAT(tag,_put)
4
5   typedef struct T *T;      /* Generic type declaration */
6
7   int map_put(T map, key_t key, value_t value) {
8     /* Insert/update key/value in map */
9   }
10
11  #undef map_put                    /* Symbolic epilogue */
12  #undef T
```

- Except for the preprocessor markup, this *generic* code looks an awful like plain C!

Common pattern: Templating + polymorphism

We better acknowledge our map may only be used as a set (no `value_t` would exist).
Lines 7–9 of the previous slide may then become:

```
1     #ifdef value_t
2     int map_put(T map, key_t key, value_t value) {
3       /* Insert or update key/value mapping in map    */
4     #else
5     int map_put(T map, key_t key) {
6       /* Try to insert key (element) in map (set)     */
7     #endif
8
9     /* Code that only relies on the key (most of it!) */
10
11    #ifdef value_t
12    /* Do the only difference due to having a value_t */
13    #endif
14
15    }
```

Common pattern: Templating + polymorphism (cont'd)

Obviously, polymorphism also extends to the definition of C `struct`'s:

```
1   #define mapping_t CAT(tag,_mapping_t)
2
3   struct {
4     key_t   key;
5   #ifdef value_t
6     value_t value;
7   #endif
8   } mapping_t;
9
10  /* Don't forget to #undef mapping_t at the end! */
11  #ifdef value_t
12  #undef tag
13  #endif /* If coming from set.h, will be #undef'd there */
```

- Here, no definition for `value_t` means using the map as a set!

Common pattern: Templating (true code reuse)

Now suppose `set.h` is to be built from any structure providing `put()`/`get()`:

```
1   #define T          CAT(tag,_t)
2   #define set_put    CAT(tag,_put)
3
4   #include <structure/map.h> /* Get map_put(), no value_t */
5   /* Or use conditional inclusion for even more backends! */
6
7   /* Since tag is the same (it is inherited in map.h), both
8      - set_put resolves to histogram_put in set.h (here) and
9      - map_put resolves to histogram_put in map.h => Profit!
10     Hence, only set ops remain to be described:
11   */
12
13  #define set_union CAT(tag,_union)
14
15  T  set_union(T set_A, T set_B) {
16  }
```

Common pattern: Packaging

`static` functions are *not visible* outside their file (their *Translation Unit*).

- Perfect for small, local functions of a greater, public code.

A function that lacks the `static` qualifier is visible from outside.

- Library code (we need *access* to the `cos`/`exp`/*etc.* symbols from the math library.)

Forward prototypes declaration is either:

- sensible (`static`):
  Some of these functions may be mutually recursive. We've got that covered.
- or mandatory (library):
  Like in a `.h` header file of the kind you were used to before.

Common pattern: Packaging (cont'd)

Our single `map.h` must take provision for both use cases (polymorphism removed):

```
1   #ifndef ds_interface
2   #define ds_interface /* Default: visible functions    */
3   #endif
4
5   /* First part: forward prototype + type declarations. */
6   ds_interface int map_put(T map, key_t key, value_t value);
7
8   /* Second part, only if we actually need the code:    */
9   #if !defined(export_interface)
10  ds_interface int map_put(T map, key_t key, value_t value) {
11    /* Blah. */
12  }
13  #endif
14
15  #undef ds_interface
```

Common pattern: Packaging (now unboxing!)

Going back to our example, actual map code is only needed in `histogram.c`:

```
1  #include <stddef.h>  /* size_t */
2  #define key_t   char*
3  #define value_t size_t
4  #define tag     histogram
5  #include <structure/map.h>
6
7  double histogram_IQR(histogram_t histogram) {
8    /* Blah. */
9  }
```

- Observe that the map type gets defined there, so we do *not* include `histogram.h`.

Common pattern: Include guard

Now for `histogram.h`, assuming at least one map function must be public:

```
1  #ifndef HISTOGRAM_H
2  #define HISTOGRAM_H
3
4  #include <stddef.h>  /* size_t */
5  #define key_t    char*
6  #define value_t  size_t
7  #define tag      histogram
8  #define export_interface /* No code needed this time */
9  #include <structure/map.h>
10
11 /* Additional functions: */
12 double histogram_IQR(histogram_t histogram);
13
14 #endif /* HISTOGRAM_H guard */
```

The guard prevents redefining the symbols (which is illegal) upon multiple inclusions.

Common pattern: Local inlining

A `static inline`-qualified function *may* be inlined by the compiler:

- If its body is worth copy-pasting into the caller, we save the cost of function call.
- Inlined code is then like macros, yet at the compiler level (with type-checking!)

Extremely useful to get the fastest maps:

```
1  static inline int wordptr_compare(const void *_left, const
       void *_right) {
2    return strcmp(*(char**)_left, *(void**)_right);
3  }
4
5  static inline size_t word_hash(char *w) {
6    size_t h = 1;
7    for (size_t a = 31 ; *w ; w++ , a *= 31) h += a**w;
8    return h;
9  }
```

Common pattern: Local inlining (cont'd)

Some of our map functions *themselves* may be small.

- For benchmarking, we therefore take advantage of the `ds_interface` option.

After `wordptr_compare` and `word_hash`, finish instantiating the fastest possible code:

```
1  #define key_t          char*
2  #define value_t        size_t
3  #define key_compare    wordptr_compare
4  #define key_hash       word_hash
5  #define ds_interface   static inline
6  #include <structure/map.h>
```

If our comparison and/or hash functions are small enough, then we'll have fast code.
Like, *really* fast.
Once we've tuned one or two more things!

Preprocessor recap

Combined, the previous techniques allow for:

1. Type-independent (generic) programming;
2. Generation of the fastest possible code for each actual types;
3. True reuse and seamless extension of generic code;
4. Arbitrarily complex data structure implementations;
5. Sensible user interface.

To get them right at the same time for some real code is tedious (but not really hard). So we give the 1200 lines of `map.h`.

- You now have all the cards to understand them.

Of course, map tests and benchmarks have all been templated too.

True preprocessor madness

What precedes is purely "standard practice" in advanced (and embedded) C.

- More advanced use of the preprocessor may be leveraged for even more generality.
- Arguably one of the most extreme pushes along these lines is the `M*LIB` library.
  - Even programs get abstracted;
  - Many (much) more involved techniques inside.

Outline

Hashing (for data indexing)

Setting the stage
- Preprocessor
- Debugging
- Error handling
- Memory management

Basic tools for data structures

Maps

Testing and benchmarking

TODO

Separation of concerns

- A search procedure in a heap that ends up exploring a negative backend array index has a *broken invariant* (a proper bug);
- A file that cannot be found because the user misspelled the name is treated by *explicit error handling*.

Bugs must be detected as early as possible and cause the program to abort.

- Encompasses failed pre-/post-conditions (*incl.* failed allocations).

Errors might be recoverable, should have a dedicated interface.

Standard debugging

Debugging is essentially about actually enforcing your code *invariants*.

- By default, code is compiled in debug mode
- Release mode: `#define NDEBUG` (well-known preprocessor switch)

Code that is explicitly executed when not in release mode:

- `assert(invariant);` (see `man 3 assert`)
  - More useful variant: `assert(invariant && "Bug reason")`
- With explicit preprocessor markup:

```
1       #ifndef NDEBUG
2       fprintf(stderr, "Searching at index: %d\n", idx);
3       #endif
```

Powerful debugging

- By default, you want all your invariants be checked;
- But you also want to turn on/off extra debugging for the specific module `foo`.

Use a `DEBUG_FOO` preprocessor switch symbol:

- Un/comment `DEBUG_FOO` definition to de/activate extra debugging code.
- Plenty of examples below.

Powerful debugging (example)

Use builtin `__func__` and `__LINE__` symbols.
Then in some function belonging to `src/foo.c`:

```
1   #include <foo.h>
2   // #define DEBUG_FOO // Extra debug switch
3
4   void foo_bar(foo_t foo) {
5
6   #if !defined(NDEBUG) && defined(DEBUG_FOO)
7     fprintf(stderr, "%s:%d: Arg foo: %p\n", __func__,
        __LINE__, foo);
8   #endif
9     /* Check pre-condition independently of DEBUG_FOO */
10    assert(foo!=NULL && "NULL argument for foo_bar");
11
12    /* Actual code for foo_bar */
13  }
```

Outline

Hashing (for data indexing)

Setting the stage
- Preprocessor
- Debugging
- Error handling
- Memory management

Basic tools for data structures

Maps

Testing and benchmarking

TODO

Standard error handling

```
1   #include <stdio.h>  /* fopen()    */
2   #include <errno.h>  /* errno      */
3   #include <string.h> /* strerror() */
4
5   FILE *fun(char *filename) {
6     FILE *fp;
7
8     fp = fopen(filename,"rt");
9     if (NULL==fp) {
10      fprintf(stderr, "%s:%d: fopen(%s): %s.\n", __func__,
            __LINE__, filename, strerror(errno));
11      return NULL;
12      /* Caller only knows about an error, not which one */
13    }
14
15    return fp;
16  }
```

Standard error handling (discussion)

- Nice, system-wide interface
- But not all functions will modify `errno`
- Errors (*and their cause!*) do not propagate easily
- Applicative code needs a more streamlined abstraction

General solution: unified type for value *or* error (and cause)

- Kernel-land trick: use reserved address range to encode error codes as pointers
- Below: user-land proposal (portable), as in Rust/OCaml/*etc.*

Powerful error handling

Applicative code essentially manipulates pointers:

```
1    typedef struct {
2      int   error;
3      void *data;
4    } rval_t;
```

- `error==0` indicates success, with (possibly unused) actual return value in `data`;
- `error>0` is an error code, with unused `data` (reuse/extend `errno` error codes!)

```
1    rval_t rval(int error, void *data) {
2      return (rval_t){.error=error, .data=data};
3    }
4    #define OK(data)       rval(0,data)
5    #define ERR(errno)     rval(errno,NULL)
6    #define IS_ERR(rv)   ((rv).error > 0)
7    #define RVAL(rv)      (rv).data
8    #define RCODE(rv)     (rv).error
```

Powerful error handling, with optional return flow tracing

```
1   // #define DEBUG_RETFLOW /* Uncomment to print returns */
2   typedef struct {
3     int  error;
4     void *data;
5   } rval_t;
6
7   #if !defined(NDEBUG) && defined(DEBUG_RETFLOW)
8   rval_t rval(int error, void *data, char *func, int line);
9   #define OK(data)      rval(0,data,__func__, __LINE__)
10  #define ERR(errno)    rval(errno,NULL,__func__,__LINE__)
11  #else
12  rval_t rval(int error, void *data);
13  #define OK(data)      rval(0,data)
14  #define ERR(errno)    rval(errno,NULL)
15  #endif
16  /* Predicate IS_ERR and accessor RVAL keep the same */
```

Powerful error handling, with optional return flow tracing (cont'd)

```
1   rval_t rval(int error, void *data
2   #ifndef NDEBUG
3                , char *func, int line
4   #endif
5                ) {
6
7   #if !defined(NDEBUG) && defined(DEBUG_RETFLOW)
8   #include <stdio.h>
9     fprintf(stderr, "[%16s:%d]␣␣return␣", func, line);
10    if (error>0) fprintf(stderr, "ERROR(%d)␣", error);
11    fprintf(stderr, "\n");
12  #endif
13
14    return (rval_t){.error=error, .data=data};
15  }
```

Outline

Hashing (for data indexing)

Setting the stage
- Preprocessor
- Debugging
- Error handling
- Memory management

Basic tools for data structures

Maps

Testing and benchmarking

TODO

Standard memory management

```
1   #include <stdlib.h>
2
3   int main(void) {
4     void *ptr = malloc(42);
5
6     free( ptr );
7
8     printf("ptr=%p\n", ptr);
9
10    exit(EXIT_SUCCESS);
11  }
```

- Fine-grained allocations only
- Should check for `ptr!=NULL`
- Not easy to trace, debug, nor extend
- `ptr` keeps its value after the call to `free()`

Powerful memory management: allocators

- Store allocator info before block.
  - Default `NULL` allocator maps to standard `malloc()`/`free()` allocator.
- Enforce `NULL` after deallocation.

```
1  // #define DEBUG_MEM /* Uncomment to debug mem de/allocs */
2
3  typedef struct {
4    void   *state; /* Allocator internal data */
5    void *(*allocate)(size_t size, void *state);
6    void  (*deallocate)(void **object_ptr, void *state);
7  } allocator_t;
```

Ultimately, make any object *propagate* its allocation policy:

- Allocate rare objects with `NULL` allocator;
- Draw tree nodes from *e.g.*, an arena allocator
  - Coarse-grained memory management useful for writing parsers, games, or GUI;
  - Key idea: delete all objects (AST subtrees, GUI widgets, *etc.*) in arena *in* $O(1)$ *time*.

Powerful memory management: allocators (interface)

```
1   #if !defined(NDEBUG) && defined(DEBUG_MEM)
2   #include <stdio.h>
3   void   *alloc__(size_t size, allocator_t *allocator, char *
        fun, int line);
4   #define alloc(size,al) alloc__(size,al,__func__, __LINE__)
5   void    dealloc__(void **objp, char *fun, int line);
6   #define dealloc(objp)  dealloc__(objp,__func__, __LINE__)
7   #else
8   void   *alloc__(size_t size, allocator_t *allocator);
9   #define alloc(size,al) alloc__(size,al)
10  void    dealloc__(void **objp);
11  #define dealloc(objp)  dealloc__(objp)
12  #endif /* Augment alloc()/dealloc() with shortcuts: */
13  #define block(sz) alloc(sz, NULL)
14  #define new(obj)  block(sizeof(*obj))
15  #define like(obj) alloc(sizeof(*obj),((allocator_t*)obj)-1)
16  #define del(objp) dealloc((void**)(objp))
```

Powerful memory management: allocators (alloc)

```
1   void   *alloc__(size_t size, allocator_t *al
2   #if !defined(NDEBUG) && defined(DEBUG_MEM)
3                   , char *func, int line
4   #endif
5                  ) { /* Add space for pointer to allocator */
6     allocator_t *ptr = al ?
7                   al->allocate(size+sizeof(ptr),al->state) :
                       calloc( 1, size+sizeof(ptr) );
8     assert( ptr && "Alloc␣failed!" ); /* Univ. precond. */
9     *ptr = al; /* Propagate allocator info */
10
11  #if !defined(NDEBUG) && defined(DEBUG_MEM)
12    fprintf(stderr, "[%16s:%d]␣␣alloc␣%zu␣bytes␣at␣%p\n",
          func, line, size, ptr+1);
13  #endif
14    return ptr+1; /* Return address *after* allocator info */
15  }
```

Powerful memory management: allocators (dealloc)

```
1   void   *dealloc__(void **objp
2   #if !defined(NDEBUG) && defined(DEBUG_MEM)
3                       , char *func, int line
4   #endif
5                     ) { /* Recover allocator info from objp */
6     if (objp && *objp) {
7       allocator_t **alp = ((allocator_t**)*objp)-1;
8   #if !defined(NDEBUG) && defined(DEBUG_MEM)
9       fprintf(stderr, "[%16s:%d]␣␣dealloc␣at␣%p\n", *
            object_ptr, fun, line);
10  #endif
11      if (*alp)
12        (*alp)->deallocate(alp, (*alp)->state);
13      else free(alp);
14      *objp = NULL; /* No silver bullet, yet sane overall */
15    }
16  }
```

Powerful memory management: extensions

1. In `de/alloc__()`, add global number and total size of live memory blocks
   - Monitor program memory usage, report statistics
2. In `de/alloc__()`, add global *set* of live memory blocks
   - Possibly with function/line info;
   - Program can reflect on the memory it uses;
   - Main application: interned (constant, unique) strings (Python VM);
   - Cosmetic application: no leaks on exit (register cleanup handler with `atexit(3)`)
3. Specialized, fast allocators for objects of known, constant size
   - List links, tree nodes, *etc.* (we'll do that for maps!)

Overall goal: minimize number of calls to the default system allocator

- Ultimately, any allocator will have to use it anyway!
- List of reusable blocks, *etc.*

At least, the infrastructure is ready for all of this.

Recap

All of the following also comes with switchable, extra debugging capabilities:

- Algorithms must check their invariants with `assert()`;
- Procedural code uses `rval_t` for control management;
- Either you get a block from specified allocator, or program aborts.

Key techniques used:

- Ternary `if`-`then`-`else` operator: `test ? consequence : alternative`
- Build on top of standards (`NDEBUG`, `errno`), seek retro-compatibility
- Preprocessor (combined) switches, predefined `__func__` and `__LINE__` macros
  - `__FILE__` also available for more fancy bloat;
  - Replace the `fprintf(stderr, ...)` by the colorful `INFO()`, `WARNING()`, *etc.* macros in `common/logging.h`;
- Guarded access to pointers: `if (foo)foo->bar = 0;`
- Block metadata: adding allocator pointer *before* returned address (more below).

Example (load file contents into memory)

```
1   rval_t file_contents(char *filename, size_t *size) {
2     FILE        *fp;
3     char     *bytes;
4     struct stat sb;
5     assert(filename && "Missing␣filename" );
6     if (-1==stat(filename, &sb)) return ERR(errno);
7     if (!(fp = fopen(filename, "rt"))) return ERR(errno);
8     bytes = block(sb.st_size);
9     for ( size_t r = 0, rr = 0 ; r < sb.st_size ; r += rr ) {
10      rr = fread(bytes+r, sb.st_size-r, 1, fp);
11      if (ferror(fp)) { del(&bytes); fclose(fp);
12        return ERR(ENODATA);
13      }
14    }
15    fclose(fp); if (size) *size = sb.st_size;
16    return OK(bytes);
17  }
```

Example (using it)

```
1   int main(int argc, char *argv[]) {
2     size_t   length;
3     char  *filename = argc>1 ? argv[1] : NULL;
4     rval_t contents = file_contents( filename, &length );
5
6     if (IS_ERR(contents)) {
7       fprintf(stderr, "%s: FAILED (error %d).\n", filename,
          RCODE(contents));
8       exit(RCODE(contents));
9     }
10
11    printf("%s: loaded %zu bytes.\n", filename, length);
12    del(&RVAL(contents));
13
14    exit(EXIT_SUCCESS);
15  }
```

Example (running it)

With full debug trace on error, watch error code propagate to the terminal:

```
$ make test/0-core@check
./test/0-core -v
[   file_contents:16]  return ERROR(2)
-v: FAILED (error 2).
make: *** [.Makefile.inc:233: test/0-core@check] Error 2
$ echo $?
2
```

Same on success:

```
$ ./test/0-core ./test/0-core.c
[   file_contents:21]  alloc 491 bytes at 0x515000000308
[   file_contents:33]  return
./test/0-core.c: loaded 491 bytes.
[            main:19]  dealloc at 0x515000000308
```

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures
- Pointer tagging
- Iterators
- Resizable arrays

Maps

Testing and benchmarking

TODO

It's written on your (inter)face!

1. We'll have at least arrays and tables as containers (that's $2 \leq 2^3$ different types)
2. Fun fact: `calloc(3)` will always return addresses on 8-byte boundaries
   - Even on 32-bit machines, because of `double`'s.
3. Fiddle with the 3 lsb's of the data structure address to encode its type number!
   - If we have any hidden metadata, it should just align on 8-byte.
   - Already the case for our allocator pointer: it's 8-byte long! (add padding if 32-bit)
4. Enforce type number of 0 for arrays
   - So syntax `arr[idx]` keeps working, stupid!
   - Other type numbers assigned freely in `core/tagging.h`:

```
1    typedef enum { T_ARRAY, ..., T_ALL } type_t;
```

   `T_ALL` only here to readily have a symbol for the number of types:
5. Profit!

Interface

```
1   #include <stdint.h>
2
3   const char *typename(type_t T);
4
5   #define typeof(obj) ((type_t)(((uintptr_t)obj)&0x7))
6   #define TAG(obj,T)  ((void*)(((uintptr_t)obj)|(T&0x7)))
7
8   #if UINTPTR_MAX == UINT64_MAX
9   #define UNTAG(tagged) \
10    ((void*)(((ptrdiff_t)(tagged))&0xfffffffffffffff8))
11
12  #elif UINTPTR_MAX == UINT32_MAX
13  #define UNTAG(tagged) \
14    ((void*)(((ptrdiff_t)(tagged))&0xfffffff8))
15  #else
16  #error "Unsupported␣pointer␣size"
17  #endif
```

typename()

```
1     /* Use the 'stringifyer' # preprocessor operator */
2     #define CHECK_TYPE(T) case T: return #T
3
4     const char *typename(type_t T) {
5       switch(T) {
6         CHECK_TYPE(T_ARRAY);
7         CHECK_TYPE(T_TABLE);
8         default: assert(T<T_ALL && "Wrong type_t");
9         return NULL; /* Silence compiler warning */
10      }
11    }
12
13    #undef CHECK_TYPE /* Clean macros namespace */
```

Example

In src/ds/iterator.c, we have something like:

```
1   iterator_t in(void *tagged) {
2
3     switch(typeof(tagged)) {
4       /* Dispatch to relevant handler, expand list */
5       case T_ARRAY: return array_forward(tagged);
6       /* Other cases as required by the situation! */
7
8       default:    /* Dummy, always-false condition */
9       assert(typeof(tagged)>T_ALL && "Wrong object type" );
10      return NULL;    /* Silence compiler warning! */
11    }
12  }
```

Pointer tagging, discussion

- If you need more types
  - Use `posix_memalign()` + `return memset(ptr,0,size)`
  - `posix_memalign` has alignment argument (reserve 4, 5, *etc.* bits for a `type_t`)
  - Alternative: `type_t` stored alongside allocator pointer in hidden metadata
- If you need dynamic types (then you're doing a virtual machine!)
  - Use `T_ALL-1` for new, generic `T_OBJ`
  - A `T_OBJ` pointer must be followed through to its real type:

```
1          typedef struct {
2            type_t T; /* Real type_t */
3            /* Other T_OBJ fields */
4          } object_t;
```

  We'll adapt this trick to specialize map backends!
  - `typeof()` becomes a function.
  - Possibly add a layer of *NaN-boxing* for scalars
    - Not in this course. Very much not-so-portable fun.

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures
- Pointer tagging
- Iterators
- Resizable arrays

Maps

Testing and benchmarking

TODO

Rationale

Suppose you iterate the slot addresses of some `array` forward:

```
1    size_t idx , len = array_length(array);
2    for ( idx = 0 ; idx < len ; idx++ ) {
3      void *obj = array + idx;
4      /* Do something with obj address */
5    }
```

Let's evolve this pattern to make it reusable.

- Make `idx` and `len` local variables of the `for` loop:

```
1        for ( size_t idx = 0 , len = array_length(array) ;
2              idx < len ; idx++ ) {
3          void *obj = array + idx;
4          /* Do something with obj address */
5        }
```

Rationale (cont'd)

- Make `obj` an outside symbol, update it in test before entering loop body:

```
1       void *obj;
2       for ( size_t idx = 0 , len = array_length(array);
3             idx < len && obj = array + idx ; idx++ ) {
4         /* Do something with obj address */
5       }
```

Why it's nice: The loop body is now completely empty. Looping logic is above.

- In effect, we want to cast the above behavior into the most general form:

```
1       relevant_itervar_t obj;
2       for ( iterator_t it = structure_iterator(structure) ;
3             next(&it, &obj) ; ) {
4         /* Do something with obj */
5       }
```

If `next()` returns strictly positive status, `obj` contains the next iterated object.

Target syntax (iteration)

We want to be able to choose the name of the current object in the iteration.

- Each iterator will obviously expect its dedicated iteration variable type.

Remember we dispatch to `array_forward()` by pointer tagging from `in()`.
In the end, we'd like to be able to write:

```
1    int *intp;
2    for each(&intp, in(array)) printf("int: %d\n", *intp);
```

Or even iterate combinatorial structures as simply as:

```
1    int perm[6];
2    for each(perm, permutations(4,6)) {
3      /* Do something with current perm[0-3] indices */
4    }
```

- Used extensively in the test template!

Target syntax (search result)

Very often, one not only needs to know whether an object was found:

```
1     iterator_t found = map_get(map, key);
2
3     if (found.status>0) { /* Key has been found */
4
5       /* Can even iterate mappings from that of the key! */
6     }
```

- *Must* be allocated *fast* (hence, on the stack!)

Definition of iterators

Fundamentally, an iterator has internal iteration state and a callback for `next()`:

```
typedef struct iterator_t {
  int    status; /* error<0, 0==finished, next found>0 */
  void  *ds; /* Having one pointer near is always nice */
  char   on_stack[ITER_STACK_STATE]; /* Iterator state */
  /* Actual iteration callback:                        */
  int  (*next)(struct iterator_t *it, void *itervarp);
} iterator_t;
```

To polish our iterator interface, we now obviously have to write:

```
#define next(itp,itervarp) \
    (((itp)->status = (itp)->next(itp,itervarp))>0)

#define each(itervarp,iter) \
    ( iterator_t it=iter ; next(&it,itervarp) ; )
```

Writing iterator `next()` callbacks

- Now only two functions are needed to get any iteration pattern for any structure!

The last trick is to cast the flat space on stack to any actual internal iterator state.
The first function to write is the (`static`) `next()` callback:

```
1   static int foo_iter_next(iterator_t *it, void *itervarp) {
2     struct {
3       /* Cast flat state stack space to arbitrary state    */
4     } *state = it->on_stack;
5
6     /* (1) Test end-of-iteration, return 0 if finished     */
7
8     /* (2) Evolve *itervarp with currently iterated object */
9
10    /* (3) Evolve state for the next iteration, return 1   */
11  }
```

Consequence: The constructor must point a new iterator to its first object (if it exists).

Writing iterator constructors

The second function, which must be public, is the iterator constructor:

```
1     iterator_t foo_iter(void *tagged) {  /* Iter constructor */
2       iterator_t it;
3       struct {
4         /* Cast flat state stack space to arbitrary state    */
5       } *state = &it.on_stack;
6
7       it.next = foo_iter_next; /* Set up callback for next() */
8
9       /* (1) Init iteration state from 'tagged' parameter    */
10
11      /* (2) Init it.status accordingly                      */
12
13      return it;
14    }
```

Iterator recap

1. Surprisingly useful as a generic search return value;
2. Syntax allows to write many algorithms just like in the textbook;
3. Any structure may be iterated in this framework;
   - Some iterators will allocate on the heap anyway (breadth-first on graphs);
   - Add cleanup callback, provide syntax for explicit iteration cancellation.
4. We also provide *generators* to iterate a few useful (combinatorial) structures:
   - Integer `range()`, floating point `linspace()`
   - Lexical `combinations()`
   - Lexical or random `permutations()`
5. If an iterator needs be passed to another function, add a thin encapsulation layer:

```
1       iterator_t *iterator(iterator_t it) {
2         return memcpy(block(sizeof(it)), &it, sizeof(it));
3       }
```

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

- Pointer tagging
- Iterators
- Resizable arrays

Maps

Testing and benchmarking

TODO

Target syntax

- Size known at array creation, your ordinary array needs no resize:

```
1    int *arr = array_of(N, int);
2
3    for (int i = 0 ; i < N ; i++) arr[i] = i;
4
5    array_delete(&arr);
```

- Unknown size at array creation, just push/pop—triggering resizes automagically!

```
1    int *arr = NULL; /* You've read it correctly */
2
3    for (int i = 0 ; i < N ; i++) array_push(&arr, &i);
4
5    array_delete(&arr);
```

- We cannot `array_push()` to a fixed-size array (like that of the first example!)

Resizing rationale

- A `dynamic` array is initialized with zero length, otherwise it's a fixed-size array;
- Calling `calloc()` is deemed costly (it is);
- Idea: allocate more array slots (`capacity`) than actually used (`length`)
  - Amortized heap allocation: use geometric (*e.g.* 1.3) array `capacity` progression
- When pushing, first check whether resize is needed (`length==capacity`)
- Pass array slot size (`stride`) to appropriate functions

```
1    typedef struct {
2      struct {
3        int      dynamic;
4        int      stride;
5      }          prop;
6      size_t     capacity;
7      size_t     length;
8    } array_hdr_t; /* Array header is hidden metadata */
```

Core interface

- A resize will generally change the array (address), so pass its address!
- Silently pass `stride` to `array_push()` to call constructor if needed.

```
1   void   *array__(size_t stride,size_t len,allocator_t *al);
2   #define array_of(N,T) array_new__(sizeof(T),N,NULL)
3   #define array(T)      array_of(0,T)
4
5   void    array_delete__(void **taggedp);
6   #define array_delete(taggedp) \
7       array_delete__((void**)(taggedp))
8
9   size_t  array_push__(void **taggedp,void *obj,size_t sz);
10  #define array_push(taggedp,obj) \
11      array_push__((void**)(taggedp),obj,sizeof(*obj))
12
13  size_t  array_pop__(void **taggedp,void *obj,size_t sz);
14  #define array_pop(taggedp,objp) \
15      array_pop__((void**)(taggedp),objp,sizeof(*objp))
```

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps
- Accomodating many data structures
- Map candidates

Testing and benchmarking

TODO

Definitions

- A *map* is a collection of unique key/value associations (*mappings*);
- A map in which mappings have no value is a *set* (its elements are the map keys);
  - Use sets for benchmarking maps (no value processing overhead)
- A mapping is stored in a *bucket*
  - Logical storage, implementation-dependent (tree node, hashtable collision list, *etc.*)
- A bucket corresponds to one or more (ordered) *slots*
  - Physical storage, implementation-independent (slot *store* provided by the map)
  - A bucket is designated by its first slot

Interface

- Creation/deletion
- Lookup key
- Insert new key/value mapping, or update value of existing key
- Remove key/value mapping

Maps in practice

- Countless applications
  - DB, names to phone numbers, parameters and their value, symbol tables, *etc.*
- Operational constraints may vary:
  1. *Track insertion order!* (which everyone assumes, but is not trivial)
  2. More often than not, data is inserted in some (quasi-)order
  3. Many inserts/updates/lookups, less removals (typical DB: 20% removals during init)
  4. Available memory: automatically adjust with `map_resize()`
  5. Bounded worst-case lookup time (hard!)
- The only better map is that which can accomodate more situations
  - Have a selected set of options to build tailored map code!
- Track insertion order w/ unbounded #insert array (`map->store.order`)
  - `LinkedHashmap` (Java) too specific, too slow, wastes too much memory
  - Only need to mark erasures (with socalled *tombstones*)
  - Rebuild `map->store.order` on actual `map_resize()`, filtering out tombstones
- Insertion order is the map iteration order (only skip tombstones)

Memory models

- Physical storage is a map responsibility (`store_insert()`/`store_remove()`)
- Most maps in existence will rarely store billions of mappings.
  - You're wasting 32 bits / pointer on a 64-bit machine
  - End of cheap memory era (AI demand + geopolitics)
- Internally replace plain C pointers with customizable `index_t` (and markers)

```
1   #include <stdint.h>
2   #ifdef ds_huge
3   typedef uint64_t         index_t;
4   #define INDEX_EMPTY      0xffffffffffffffffull
5   #define INDEX_TOMBSTONE  0xfffffffffffffffeull
6   #else /* 32-bit internal pointers by default */
7   typedef uint32_t         index_t;
8   #define INDEX_EMPTY      0xffffffffu
9   #define INDEX_TOMBSTONE  0xfffffffeu
10  #endif
```

Memory models (cont'd)

- Special out-of-band markers semantics:
  - `INDEX_EMPTY`: "has never been used so far"
  - `INDEX_TOMBSTONE`: "has been erased"
  - Any other value (starting from 0) is an index in any internal map array

```
1  #define index_is_empty(idx)    ((idx)==INDEX_EMPTY)
2  #define index_is_freed(idx)    ((idx)==INDEX_TOMBSTONE)
3  #define index_is_free(idx)     ((idx)>=INDEX_TOMBSTONE)
4  #define index_free(idxp)      (*(idxp)=INDEX_TOMBSTONE)
```

As a consequence, fastest reset/init of any `index_t` array in a map with:

```
1  memset(array, 0xff, array_length(array)*sizeof(*array));
```

Applies to:

- Resizing the insertion order array `map->store.order`
- Node "pointers" in recursive data structures (empty list/tree, *etc.*)

Slot store

Our slot store will maintain the list of slots that are currently free:

```
1   #define freelist_is_empty(store) \
2       index_is_empty((store).free_list)
3
4   typedef struct {
5     slot_t      *slot;         /* Storage area (flat array) */
6     index_t      free_list;    /* Free slots list head!     */
7     size_t       nfree;        /* Accessed often            */
8
9     index_t     *order;        /* Track insertion order     */
10    size_t       ntombstones;  /* Track sparseness measure  */
11  } store_t;
```

- At store init, put all slots of resizable array `slot` in the free list;
- `store_insert()` pops from the free list, `store_remove()` pushes to it;
  - *Very* fast $O(1)$ operations: free list only used as a stack;
- Kind of a dedicated, internal arena allocator (only supporting slots.)

Slot design

- A `slot_t` will obviously store a mapping:

```
1       typedef struct {
2         key_t   key;    /* maps and sets */
3       #ifdef value_t
4         value_t value; /* maps only     */
5       #endif
6       } mapping_t;
```

- To fully accomodate the upstream map requirements, a `slot_t` also must feature:
  1. Some `index_t next_free` info for the next slot in the free list;
  2. An `index_t pos` to point in the `store.order` array.
- The rest depends on the actual backend:
  - Hashtable (`ds_backend_table`) *vs.* some recursive structure (`ds_backend_nodes`);
  - Whether it uses a hash value (`ds_backend_uses_hash`)
    Hashtables obviously do, yet also some tree structures (hashtries, treaps).

Slot design (cont'd)

```
1   typedef struct {
2     mapping_t kv;     /* actual key/value storage!   */
3   #ifdef ds_backend_uses_hash
4     hash_t    hash;   /* store hash close to the key */
5   #endif
6     index_t   pos;    /* index into insertion order  */
7   #if defined(ds_backend_table)
8     union {
9       index_t next_free;       /* When slot is free */
10      index_t next_chain;      /* Closed addressing */
11      index_t psl;             /* Open addressing   */
12    }         meta;
13  #elif defined(ds_backend_nodes)
14    index_t   node[1<<ds_backend_nodes] /* Pointers */
15  #endif
16  } slot_t;
```

Slot design (discussion)

For hashtables, we use a `union` (`meta`) to have different names for the same `index_t`, *depending on the current usage of the slot*:

- If it's free, this index is called `next_free`, it is that of the next slot in the free list;
- Otherwise, it is called either:
  - `next_chain`: Index of the next slot in a collision list (closed addressing), or
  - `psl`: Some valuable info (open addressing, more later.)

For recursive structures, we only need some $2^{\texttt{ds_backend_nodes}}$ array of `index_t` pointer(s) to the next slot(s) in the structure:

- If `ds_backend_nodes` is 0, we have one pointer and the structure is a list;
- If it is 1, we have two pointers for a binary tree (BST, treap, basic hashtrie);
- It it is bigger, the tree has (exponentially) higher arity (general hashtries.)

Now for the neat trick: In either case, at least one `index_t` exists (either `meta` or `node[0]`), so free slots are handled uniformly identically.

A universal design for maps

Then any map is some logical structure built upon physical storage:

```
1   struct T {
2     map_header_t hdr;   /* Length, callbacks, etc.          */
3     map_stats_t  stats; /* Instrumentation (benchmarking)   */
4
5     store_t      store; /* Storage + insert order tracking */
6
7     index_t     *bucket;   /* Logical structure storage     */
8     size_t       dim;      /* Logical structure dimension   */
9
10    uint8_t     *control;  /* Support ultra-fast lookups    */
11  };
```

All maps have `dim` logical `bucket`'s:

- Hashtables will use (special) primes or powers-of-two for `dim`;
- Recursive structures all keep `dim==1` (one list head, one tree root.)

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps
- Accomodating many data structures
- Map candidates

Testing and benchmarking

TODO

Pseudocode for maps: `map_get()`

- This one is merely boiler-plate code around iterators.

```
1    iterator_t map_get(T map, key_t key) {
2      iterator_t notfound = { 0 };
3      index_t    slot;
4
5      if (impl_lookup(map, key, &slot))
6
7        return map_iter(map, map->store[slot].pos);
8
9      else return notfound;
10   }
```

Pseudocode for maps: `map_remove()`

- This one is pretty self-explanatory.

```
1   size_t      map_remove(T map, key_t key) {
2     index_t     slot;
3
4     if (impl_remove(map, key, &slot)) {
5
6       size_t pos = map->store.slot[slot].pos;
7       map->store.order[pos] = INDEX_TOMBSTONE;
8
9       store_remove(map->store, slot);
10      map->hdr.length--;
11      map_resize(map);
12    }
13
14    return map->hdr.length;
15  }
```

Pseudocode for maps: `map_put()`

- Recursive structures will find a new bucket *inside* slots
  - The reason we work with `index_t *slot` (and for your headaches during resizes!)

```
1   size_t      map_put(T map, key_t key, value_t value) {
2     index_t    *slot;
3
4     if (!impl_lookup_or_bucket(map, key, &slot)) {
5
6       index_t idx = store_insert(&map->store, key, value);
7
8       map_put_idx(map, idx, slot);
9     }
10    else map->store.slot[*slot].value = value;
11
12    return map->hdr.length;
13  }
```

Pseudocode for maps: `map_put_idx()`

- This one has to stand out to implement `map_resize()`
  - `map_resize()` will reuse the store as it is
  - Then the mappings already have their indices in the store

```
1   static void map_put_idx(T map, index_t idx, index_t *bkt) {
2     size_t ins_order = array_push(&map->store.order,&idx)-1;
3
4     map->store.slot[idx].pos = ins_order;
5
6     impl_insert(map, idx, bkt);
7
8     map->hdr.length++;
9     map_resize(map);
10  }
```

Implementation candidates

Any implementation backend that can provide the following makes a valid map:

- `int impl_lookup(T map, key_t key, index_t *slot);`
  Lookup `key` in `map`, return zero if not found.
  Otherwise: update `*slot` with store index of result and return non-zero.
- `int impl_remove(T map, key_t key, index_t *slot);`
  Lookup `key` in `map`, return zero if not found.
  Otherwise: delete mapping from `map` structure, update `*slot` with index of the mapping to be removed from store, and return non-zero.
- `void impl_insert(T map, index_t idx, index_t *bucket);`
  Insert in `map` bucket `*bucket` the new mapping stored at index `idx`.

Implementation candidates (cont'd)

- `int impl_lookup_or_bucket(T map, key_t key, index_t **bucketp);`
  This one is the trickiest by far!
  Lookup `key` in `map`.
  If it is found, update `*bucketp` with *address* of the slot (*in the store*) and return non-zero.
  Otherwise: update `*bucketp` with *address* of the bucket (*in the structure*) where to insert the new mapping and return zero.
  Depending on the return value, the index does not live in the same space!
- The rationale is to save as much work as possible (the lookup) for `impl_insert()`.

A theoretical design for fair comparison

What the maths say about `dim` for quadratic addressing:

- Primes that will allow for complete `bucket`'s exploration equal 3 (mod 4);
- Enforcing the same constraint for powers-of-two implies using *triangular numbers*.

Other hashtables immune to the issue, explore all buckets:

- If option `ds_ht_primes` is defined, select primes for `dim` (with 1.3 growth factor);
- Otherwise, default to powers-of-two (a choice had to be made.)

Recursive structures not affected (only one bucket.)

Map recap

- Some abstraction layer between:
  - Top: Expectations of upstream algorithms (insertion order, storage, iteration, *etc.*);
  - Bottom: The logical essence of the various ways of inserting/searching data.
- Decoupling physical/logical storage is instrumental (much code is reused);
- High-end testing ground for practical complexity assessments:
  - Integrated, templated tests and benchmarks;
  - Evaluate the effect of any backend option in any practical scenario;
- Eventually: battle-tested, modular, generic code for fastest maps/sets in plain C.

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking
- Testing
- Benchmarking

TODO

One infamous bug story

The binary search function of the Java standard library used to compute the median index (`low<=high`) like:

```
1     int mid = (low+high) / 2;
```

Bug went unsuspected for 15+ years. — *What bug? That's the exact maths!*

- Sure. But `(low+high)` will overflow as soon as `low+high > (1<<31)-1`.
- `mid` index then gets corrupted. Search fails. Critical software at risk. Worldwide.
- Patch with exact same maths, without overflowing:

```
1       int mid = low + (high-low) / 2;
```

Despite correct maths, bug appeared when machines got enough memory to trigger it.

- Trigger as much edge cases as possible during testing.

Testing rationale

Basic test typology

- Unit testing: Interface behavior
- Functional testing: Basic algorithm behavior
- Random testing: Functional testing on long streaks of random data

Functional testing with a *digital twin*

- Use simplest, yet likely slower, equivalent code for validation.
- Searching an array is simplest: can be provably correct, but runs in $O(N)$.
- Searching a Robing-Hood hashtable is more tedious
  - Test results on the table are compared to those on the simplest code for arrays.

Most reasonable way to trigger unsuspected edge cases: run code on random data

- Control RNG seed for test debugging, then use random seeds for real tests.
- Also known as *fuzz testing*.

How we'll do the testing

1. Have unit and functional tests in `test/1-map.c`
   Test template is in `test/map-test.h` .
2. Since our digital twin would be logarithmic search at best, use integers instead.
   Cheapest (constant-time), exact local ground truth.
3. Also use (possibly random) integers in the benchmark.
   That's our fuzzing-like strategy, report failed lookups.

Your basic workflow:

1. Implement map option `ds_backend_foo`
2. Check at least you did not break the basics:
   `apman$ make deep-clean && make test/1-map@check`
3. Track the existence of more subtle bugs with the benchmark on random integers
4. Verify robustness with ascending/descending integer sequences

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking
- Testing
- Benchmarking

TODO

Benchmarking maps

- Map benchmarking metrics (minimal set), per mapping:
  - Memory overhead compared to using a flat array for key/value storage, in bytes
  - Time to insert-absent/lookup-present/lookup-absent/remove-present, in nsecs
- *How many nanoseconds to find a specific needle in a mega-haystack?*
  - Micro-benchmarking unreliable, aggregate stats on multiple runs
  - Design benchmark template accordingly and report lookup errors:
    1. Insert N different integers in empty map
    2. Lookup those N integers (known to be in the map)
    3. Lookup N other integers (known *not* to be in the map)
    4. Remove the N integers
  - Save raw run stats in CSV format
- Benchmark rendering: timing plots + radar plot with Inter-Quartile Range
  - Map implementations feature a variety of behaviors.
- Have a Python script to run benchmark configs and make the plots
  - Forget `matlab`, welcome to `maplab`!

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking

TODO
- Getting started
- Map options
- Moving forward

A few demos

Directory `doc/demo` contains examples of various types of genericity for *lists*.

- These are shorter, focused snippets;
- Use them as a basis for the part of your report explaining how genericity may be implemented in C, also with their preferred use cases.
  - We give you code, you write your own course notes!
- Feel free to *not* understand everything first time.

Learning to write *about* code is just as important as writing it.

Resizable arrays (`src/ds/array.c`)

1. Study the interaction between `include/ds/array.h`, `src/ds/array.c`, and `test/1-array.c`
   - Make a diagram of the memory areas at play (header, slots), particularly during a resize operation;
   - Pay attention to pointer tagging, observe how iterators are constructed;
   - Understand where/why `memset()`, `memcpy()` and friends are useful;
   - The test summarizes most of the properties that other modules will need.
2. A few functions must be completed in `src/ds/array.c`, check whether your proposal is OK with:

   ```
   make test/1-array@check
   ```
3. Write down a recap of the techniques used: type of genericity, hidden metadata.
4. Once your arrays behave correctly, that unlocks the basic versions of a few maps.

Bottom line: You should spend much more time reading code than writing it.

- Formulating your questions is an *excellent* exercise.

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps

Testing and benchmarking

TODO

- Getting started
- Map options
- Moving forward

General map options

Make live memory scans (JTAG, shared/leaked mem.) more difficult for an opponent:

- `ds_secure_removal`
  Whether to reset mapping memory after `map_remove()`. Already implemented.
  Note: Popularized by GrapheneOS (and Molly!)

More often than not, the same key gets probed several times in a row.

- `ds_mtf`
  If a data structure uses some form of chaining, a common trick is to swap the last key found with its chain (list) head. To be used primarily with `ds_ht_chained` addressing (below.) Also useful with hashtries or coalesced designs like Verstable.
  Note: Also called *Ryabko's bookstack*, instrumental in the `bzip2`/`xz` compressors.

All data structures involving a hash may use the following options:

- `hash_no_finalize`: Disable the (Fibonacci) hash finalizer. Already implemented.
- `hash_filtered`: Skip call to equality test if hashes mismatch.

General recursive structure options

Recall that `ds_backend_nodes` is defined to be $\log_2$ of the arity.

- `ds_nd_list`: Already implemented, with sorted variant.
- `ds_nd_bst`: Binary search trees.
  All tree structures need deep understanding of the map code: working with addresses inside slots, with their store subject to resizing (and therefore address change), is challenging yet of course doable.
- `ds_nd_treap`
  Actually an option of the above: whether to use the hash for randomized balancing. Much of the BST code could be factored out.
- `ds_nd_hashtrie`: Use plain hash bits as n-ary tree routing info.
  If all hash bits are consumed, natively degenerates into list collision chaining.
  Note: Yet another elegant idea by Chris Wellons! Also check HAMT structures.

General hashtable options

All hashtables already honor the following options:

- `ds_ht_primes`
  Whether to use primes equal 3 (mod 4) instead of powers-of-two for `dim`.
  See `test/1-map.c` if you want to generate other primes.
- `ds_ht_modulo_fast`: If `dim` is a power-of-two, replace *modulo* with *and*.
  Note: Make sure to check the works of Daniel Lemire!
- `ds_ht_lf_min`, `ds_ht_lf_max`
  Bounds for the *load factor*, the ratio of #tombstones plus #mappings over `dim`.
  Used to trigger resizes on `map_put()` and `map_remove()`.
  Different backends will accomodate different ranges for the upper bound.
- `ds_ht_chained`, `ds_ht_quadratic`, `ds_ht_linear`, `ds_ht_robin_hood`
  Actual hashtable addressing backend (options are mutually exclusive.)
  Robin-Hood has been set aside *linear* in order not to clutter the latter.

Open-addressing hashtable options

Open addressing may use tombstones (that's the default, easy way to go.)

- `ds_ht_tomb_reuse`: Whether tombstones are reused.
  Virtually all open addressing variants may choose to reuse tombstones.
- `ds_ht_tomb_none`
  In the specific case of linear probing (including Robin-Hood), it is even possible to avoid tombstones completely. But it's *not* trivial!
  Note: Knuth's Algorithm R for standard linear probing may not be the one to use for Robin-Hood! The Java implementation is a good starting point.

Linear probing (but not Robin-Hood!) may try to reduce primary clustering by *salting* (randomizing) the displacement for the next probed index.

- `ds_ht_hash_salted`
  Add a few bits of the key hash in the probe displacement.
  If all hash bits have been consumed, defaults to 1 (standard linear probing.)
  Note: Trick was used until early Python 3.x series for the core dictionary structure.

Outline

Hashing (for data indexing)

Setting the stage

Basic tools for data structures

Maps

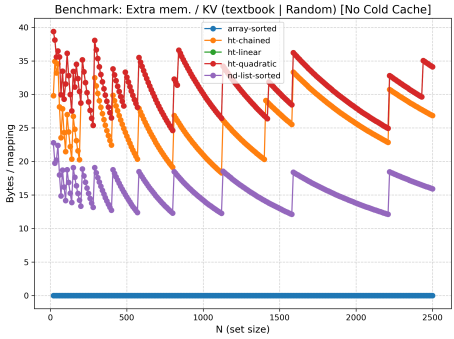
Testing and benchmarking

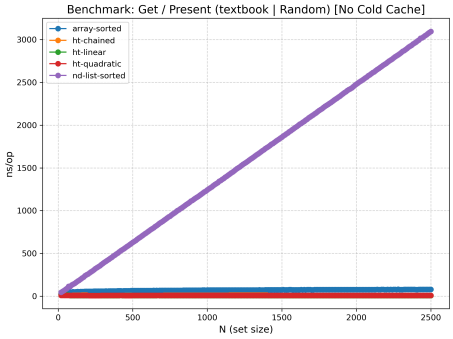
TODO

- Getting started
- Map options
- Moving forward

```
apman$ ./maplab --experiment textbook --preset tiny
```

But first, compile benchmark execs: `apman$ make`

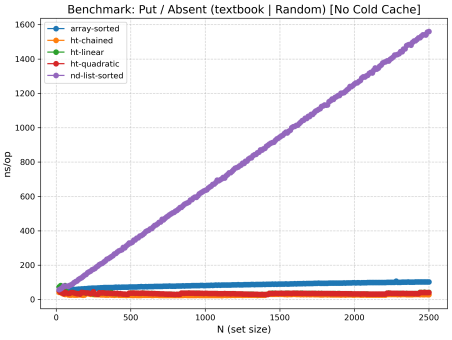


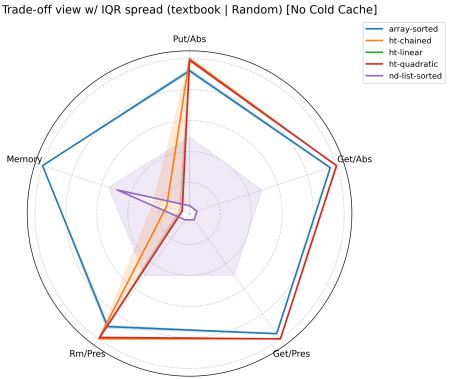


- Once you've explained these plots and you're convinced linear-time is unusable:
  1. Remove directory `experiment/textbook/output` (or rename it to keep it.)
  2. Remove `textbook-nd-list-sorted` from `experiment/textbook/benchmark`
  3. Use more data: `apman$ ./maplab --experiment textbook --preset small`

`apman$ ./maplab --experiment textbook --preset tiny` (cont'd)

For the radar plot: outer is better, solid line is median, shaded area is IQR.





- With options `ds_secure_removal`, `ds_ht_primes`, `hash_no_finalize`, `ds_ht_modulo_fast`, `ds_ht_lf_{min,max}` already implemented, you can exercise benchmarking to draw a number of conclusions *today*.

Maps

Learn how to *use* the environment and the methodology first.
All while concentrating on implementing your best maps. In higher-level C.
That should account for up to 70% of the project.

1. Implement, test and benchmark map variants in:
   `include/ds/map/backend/{chained,linear,quadratic,bst,hashtrie}.h`
   This part of your report will basically be a plain textbook on algorithms.
   Your report should ask and answer the right questions with empirical evidence.
   See the `README.md` for how to design and run map experiments.
2. Download some real serious external code
   Integrate it in the benchmarks, *as seamlessly as possible*. Explain how you did.
   Give the wonderful *Verstable* map a try!
   Quadratic addressing with coalesced chains and *relative* pointers...
   Exercise your skills in C templating. You're only a couple `#define`'s away!

Those two steps ensure you master the coding standards at a reasonable level.

Maps (cont'd)

You may also want to reflect on the methodology itself:

1. Feel free to dissect, criticize, evolve the tests or the benchmark
   Explain why and how you did (also to validate your approach.)
   For example, the benchmark cannot measure the effect of Move-to-Front.
   Also, the unit tests could be slightly more stringent.
2. Same for the map code
   You may want to handle insertion order more thoroughly (tombstones.)
   Also, it lacks proper de/allocation for the `control` bytes!

Add more (simpler!) data structures

1. Add the *deque* data structure:
   - Backends: resizable arrays *vs.* intrusive links (see)
   - Upstream abstract data types: lists, stacks, queues
   - Design the interface, the implementations, the tests and the benchmarks
   - Explain the typical use cases.
2. *Embedded specials*
   - Add a bip-buffer! Explain why it may be useful.
   - Consider the resemblance with a general arena allocator...
   - Map-related: implement Cuckoo hashing

Once you've followed the OS course, think of adding:

- A thread pool;
- Asynchronous computations with `future_t`'s
- You may then start diving into the harsh world of *concurrent* data structures...

Typical report outline

A report skeleton is given in `doc/report`.

- Pay attention to the many references you may want to explore!

We expect a discussion of the following points, backed by benchmarks where relevant:

1. Genericity: why and how?
2. Arrays: why and how?
3. Maps on unsorted data
   - Hashtables
   - Hashtries
4. Maps on sorted data
   - Search trees
   - Binary search
5. Deques
   - Design of the data structure, of the tests, and of the benchmark
   - Discussion of the sentinel technique for the link-based backend

What *we* will pay attention to

For example:

- Could your report be used as a reference for other students?
- Quality, coverage, and validation/refutation of your hypotheses (maps)
- How the techniques we introduced were adapted (deques)

We show you a (vast) landscape, show us how you explore it scientifically!